

PROJECT BASED EXPERIMENTAL LEARNING

COURSE- DATABASE MANAGEMENT SYSTEM (NoSQL / MongoDB)

PROJECT REPORT

TOPIC- MONGODB STUDENT DATABASE OPERATIONS



SUBMITTED TO- MR. Harendra Singh Rajpoot

Submitted by- Saurabh Singh

BCADS27

Roll no. - 1250258403

3rd semester

Babu Banarasi Das University

Lucknow, U.P

Acknowledgment

I would like to express my special thanks of gratitude to my mentor Mr. Harendra Singh Rajpoot, who gave me the golden opportunity to do this wonderful project on MongoDB Student Database Operations. My mentor helped me a lot in completing this project within the given time frame, and I learned many new things about NoSQL databases from him. He converted his experience into our knowledge.

Secondly, I want to show my gratitude to IBM, which provided me this project-based learning experience; it helped me to achieve my goals more efficiently.

Abstract

This report presents a MongoDB-based Student Database Management project that demonstrates the core Create, Read, Update and Delete (CRUD) operations of a NoSQL document database. A "students" collection is used to store student records containing roll number, name, age, department and marks. The dataset used in this revision of the project has been refreshed with a new set of student records (Roll No. 501-551) while the methodology and operations remain identical. The project implements single and bulk insertion, conditional updates, single and bulk deletion, and a wide range of query operations using MongoDB comparison and logical operators such as \$gt, \$lt, \$eq, \$gte, \$ne, \$and, \$or and \$not. The implementation demonstrates how MongoDB can be used to efficiently manage, filter and retrieve structured student data.

Introduction

MongoDB is a popular NoSQL, document-oriented database that stores data as flexible, JSON-like documents rather than in rigid rows and columns. This makes it well suited for applications where the data structure may evolve over time, such as academic record-keeping systems. This project focuses on building and querying a "students" collection that stores academic details of 51 students, and demonstrates the standard database operations - insertion, updation, deletion and querying - that form the backbone of any real-world database-driven application.

Methodology

The project was carried out step by step inside the MongoDB shell (mongosh), with every operation executed against a single collection named "students" in a database created for this purpose. The complete methodology, along with the exact code executed and the output produced by MongoDB at each step, is given below.

1. Insert One

Inserts a single student document into the students collection.

Code:

```
db.students.insertOne({
  rollNo: 501,
  name: "Aarav",
  age: 18,
  department: "CSE",
  marks: 61
})
```

2. Insert Many

Inserts multiple student documents (roll numbers 502 to 551) into the collection at the same time.

Code:

```
db.students.insertMany([
{ rollNo: 502, name: "Vivaan", age: 20, department: "IT", marks: 75 },
{ rollNo: 503, name: "Aditya", age: 19, department: "ECE", marks: 68 },
{ rollNo: 504, name: "Kabir", age: 18, department: "ME", marks: 94 },
{ rollNo: 505, name: "Ishaan", age: 18, department: "CSE", marks: 97 },
{ rollNo: 506, name: "Reyansh", age: 21, department: "IT", marks: 62 },
{ rollNo: 507, name: "Ayaan", age: 18, department: "ECE", marks: 65 },
{ rollNo: 508, name: "Vihaan", age: 19, department: "ME", marks: 74 },
{ rollNo: 509, name: "Arnav", age: 22, department: "CSE", marks: 98 },
{ rollNo: 510, name: "Krishna", age: 18, department: "IT", marks: 95 },
{ rollNo: 511, name: "Dhruv", age: 19, department: "ECE", marks: 94 },
{ rollNo: 512, name: "Yuvraj", age: 21, department: "ME", marks: 74 },
{ rollNo: 513, name: "Shaurya", age: 21, department: "CSE", marks: 97 },
{ rollNo: 514, name: "Rudra", age: 20, department: "IT", marks: 60 },
{ rollNo: 515, name: "Advait", age: 19, department: "ECE", marks: 87 },
{ rollNo: 516, name: "Kiara", age: 20, department: "ME", marks: 77 },
{ rollNo: 517, name: "Anaya", age: 19, department: "CSE", marks: 73 },
{ rollNo: 518, name: "Diya", age: 20, department: "IT", marks: 66 },
{ rollNo: 519, name: "Myra", age: 18, department: "ECE", marks: 84 },
{ rollNo: 520, name: "Aadhya", age: 18, department: "ME", marks: 82 },
{ rollNo: 521, name: "Saanvi", age: 20, department: "CSE", marks: 98 },
{ rollNo: 522, name: "Aarohi", age: 20, department: "IT", marks: 62 },
{ rollNo: 523, name: "Ira", age: 21, department: "ECE", marks: 94 },
{ rollNo: 524, name: "Navya", age: 18, department: "ME", marks: 84 },
{ rollNo: 525, name: "Prisha", age: 18, department: "CSE", marks: 95 },
{ rollNo: 526, name: "Riya", age: 20, department: "IT", marks: 99 },
{ rollNo: 527, name: "Siya", age: 20, department: "ECE", marks: 96 },
{ rollNo: 528, name: "Anvi", age: 19, department: "ME", marks: 64 },
{ rollNo: 529, name: "Zara", age: 18, department: "CSE", marks: 74 },
{ rollNo: 530, name: "Pihu", age: 20, department: "IT", marks: 65 },
{ rollNo: 531, name: "Om", age: 19, department: "ECE", marks: 66 },
{ rollNo: 532, name: "Rehan", age: 21, department: "ME", marks: 77 },
{ rollNo: 533, name: "Veer", age: 21, department: "CSE", marks: 83 },
{ rollNo: 534, name: "Aryan", age: 19, department: "IT", marks: 83 },
{ rollNo: 535, name: "Dev", age: 20, department: "ECE", marks: 73 },
{ rollNo: 536, name: "Ansh", age: 20, department: "ME", marks: 64 },
{ rollNo: 537, name: "Yash", age: 22, department: "CSE", marks: 70 },
{ rollNo: 538, name: "Parth", age: 22, department: "IT", marks: 75 },
{ rollNo: 539, name: "Raj", age: 19, department: "ECE", marks: 89 },
{ rollNo: 540, name: "Sameer", age: 21, department: "ME", marks: 77 },
{ rollNo: 541, name: "Meera", age: 22, department: "CSE", marks: 74 },
{ rollNo: 542, name: "Tanvi", age: 20, department: "IT", marks: 63 },
{ rollNo: 543, name: "Aisha", age: 19, department: "ECE", marks: 62 },
{ rollNo: 544, name: "Kavya", age: 20, department: "ME", marks: 85 },
{ rollNo: 545, name: "Rhea", age: 20, department: "CSE", marks: 64 },
{ rollNo: 546, name: "Avni", age: 19, department: "IT", marks: 96 },
{ rollNo: 547, name: "Gauri", age: 20, department: "ECE", marks: 73 },
{ rollNo: 548, name: "Nitya", age: 21, department: "ME", marks: 85 },
{ rollNo: 549, name: "Sara", age: 21, department: "CSE", marks: 69 },
{ rollNo: 550, name: "Vanya", age: 20, department: "IT", marks: 68 },
{ rollNo: 551, name: "Lakshya", age: 19, department: "ECE", marks: 95 }
])
```

3. Update One

Updates the first document that matches the given condition - here, the marks of roll number 501 are updated from 61 to 60.

Code:

```
db.students.updateOne(  
  { rollNo: 501 },  
  { $set: { marks: 60 } }  
)
```

Output:

```
{  
  acknowledged: true,  
  matchedCount: 1,  
  modifiedCount: 1  
}
```

4. Update Many

Updates all documents that match the given condition - here, every student in the CSE department is given a new status field set to "Active".

Code:

```
db.students.updateMany(  
  { department: "CSE" },  
  { $set: { status: "Active" } }  
)
```

Output:

```
{  
  acknowledged: true,  
  matchedCount: 13,  
  modifiedCount: 13  
}
```

5. Delete One

Deletes the first document that matches the given condition - here, the record of roll number 551 is removed.

Code:

```
db.students.deleteOne({ rollNo: 551 })
```

Output:

```
{  
  acknowledged: true,  
  deletedCount: 1  
}
```

6. Delete Many

Deletes all documents that match the given condition - here, every student with marks less than 40 is removed.

Code:

```
db.students.deleteMany({ marks: { $lt: 40 } })
```

Output:

```
{
  acknowledged: true,
  deletedCount: 0
}
```

7. Query Operations

A series of find() queries were run using MongoDB's comparison and logical operators to filter and retrieve student records.

Marks greater than 80:

Code:

```
db.students.find({ marks: { $gt: 80 } })
```

Output:

```
504 Kabir 18 ME 94
505 Ishaan 18 CSE 97
509 Arnav 22 CSE 98
510 Krishna 18 IT 95
511 Dhruv 19 ECE 94
513 Shaurya 21 CSE 97
515 Advait 19 ECE 87
519 Myra 18 ECE 84
520 Aadhya 18 ME 82
521 Saanvi 20 CSE 98
523 Ira 21 ECE 94
524 Navya 18 ME 84
525 Prisha 18 CSE 95
526 Riya 20 IT 99
527 Siya 20 ECE 96
533 Veer 21 CSE 83
534 Aryan 19 IT 83
539 Raj 19 ECE 89
544 Kavya 20 ME 85
546 Avni 19 IT 96
548 Nitya 21 ME 85
```

Marks less than 80:

Code:

```
db.students.find({ marks: { $lt: 80 } })
```

Output:

```
501 Aarav 18 CSE 60
502 Vivaan 20 IT 75
503 Aditya 19 ECE 68
506 Reyansh 21 IT 62
507 Ayaan 18 ECE 65
508 Vihaan 19 ME 74
512 Yuvraj 21 ME 74
514 Rudra 20 IT 60
516 Kiara 20 ME 77
517 Anaya 19 CSE 73
518 Diya 20 IT 66
522 Aarohi 20 IT 62
```

528 Anvi 19 ME 64
529 Zara 18 CSE 74
530 Pihu 20 IT 65
531 Om 19 ECE 66
532 Rehan 21 ME 77
535 Dev 20 ECE 73
536 Ansh 20 ME 64
537 Yash 22 CSE 70
538 Parth 22 IT 75
540 Sameer 21 ME 77
541 Meera 22 CSE 74
542 Tanvi 20 IT 63
543 Aisha 19 ECE 62
545 Rhea 20 CSE 64
547 Gauri 20 ECE 73
549 Sara 21 CSE 69
550 Vanya 20 IT 68

Marks equal to 60:

Code:

```
db.students.find({ marks: { $eq: 60 } })
```

Output:

501 Aarav 18 CSE 60
514 Rudra 20 IT 60

Marks greater than or equal to 80:

Code:

```
db.students.find({ marks: { $gte: 80 } })
```

Output:

504 Kabir 18 ME 94
505 Ishaan 18 CSE 97
509 Arnav 22 CSE 98
510 Krishna 18 IT 95
511 Dhruv 19 ECE 94
513 Shaurya 21 CSE 97
515 Advait 19 ECE 87
519 Myra 18 ECE 84
520 Aadhya 18 ME 82
521 Saanvi 20 CSE 98
523 Ira 21 ECE 94
524 Navya 18 ME 84
525 Prisha 18 CSE 95
526 Riya 20 IT 99
527 Siya 20 ECE 96
533 Veer 21 CSE 83
534 Aryan 19 IT 83
539 Raj 19 ECE 89
544 Kavya 20 ME 85
546 Avni 19 IT 96
548 Nitya 21 ME 85

No student in the refreshed dataset scores exactly 80, so this result set is identical to the strict $\$gt$ 80 query above.

Marks not equal to 60:

Code:

```
db.students.find({ marks: { $ne: 60 } })
```

This returns all 48 remaining students whose marks are not exactly 60.

Marks greater than 70 AND age less than 23:

Code:

```
db.students.find({
  $and: [
    { marks: { $gt: 70 } },
    { age: { $lt: 23 } }
  ]
})
```

Output:

```
502 Vivaan 20 IT 75
504 Kabir 18 ME 94
505 Ishaan 18 CSE 97
508 Vihaan 19 ME 74
509 Arnav 22 CSE 98
510 Krishna 18 IT 95
511 Dhruv 19 ECE 94
512 Yuvraj 21 ME 74
513 Shaurya 21 CSE 97
515 Advait 19 ECE 87
516 Kiara 20 ME 77
517 Anaya 19 CSE 73
519 Myra 18 ECE 84
520 Aadhya 18 ME 82
521 Saanvi 20 CSE 98
523 Ira 21 ECE 94
524 Navya 18 ME 84
525 Prisha 18 CSE 95
526 Riya 20 IT 99
527 Siya 20 ECE 96
529 Zara 18 CSE 74
532 Rehan 21 ME 77
533 Veer 21 CSE 83
534 Aryan 19 IT 83
535 Dev 20 ECE 73
538 Parth 22 IT 75
539 Raj 19 ECE 89
540 Sameer 21 ME 77
541 Meera 22 CSE 74
544 Kavya 20 ME 85
546 Avni 19 IT 96
547 Gauri 20 ECE 73
548 Nitya 21 ME 85
```

Department is CSE OR ECE:

Code:

```
db.students.find({
  $or: [
    { department: "CSE" },
    { department: "ECE" }
  ]
})
```

CSE and ECE department students are returned.

CSE:

```
501 Aarav    505 Ishaan    509 Arnav    513 Shaurya
517 Anaya    521 Saanvi    525 Prisha    529 Zara
533 Veer     537 Yash      541 Meera     545 Rhea
549 Sara
```

ECE:

```
503 Aditya    507 Ayaan    511 Dhruv    515 Advait
519 Myra      523 Ira      527 Siya     531 Om
535 Dev       539 Raj      543 Aisha    547 Gauri
```

Marks NOT greater than 80:**Code:**

```
db.students.find({ marks: { $not: { $gt: 80 } } })
```

Output:

```
501 Aarav 18 CSE 60
502 Vivaan 20 IT 75
503 Aditya 19 ECE 68
506 Reyansh 21 IT 62
507 Ayaan 18 ECE 65
508 Vihaan 19 ME 74
512 Yuvraj 21 ME 74
514 Rudra 20 IT 60
516 Kiara 20 ME 77
517 Anaya 19 CSE 73
518 Diya 20 IT 66
522 Aarohi 20 IT 62
528 Anvi 19 ME 64
529 Zara 18 CSE 74
530 Pihu 20 IT 65
531 Om 19 ECE 66
532 Rehan 21 ME 77
535 Dev 20 ECE 73
536 Ansh 20 ME 64
537 Yash 22 CSE 70
538 Parth 22 IT 75
540 Sameer 21 ME 77
541 Meera 22 CSE 74
542 Tanvi 20 IT 63
543 Aisha 19 ECE 62
545 Rhea 20 CSE 64
547 Gauri 20 ECE 73
549 Sara 21 CSE 69
550 Vanya 20 IT 68
```

8. MongoDB Operator Summary

Operator	Meaning	Example
\$gt	Greater than	marks: { \$gt: 80 }
\$lt	Less than	marks: { \$lt: 80 }
\$eq	Equal to	marks: { \$eq: 60 }
\$gte	Greater than or equal to	marks: { \$gte: 80 }
\$ne	Not equal to	marks: { \$ne: 60 }
\$and	All conditions must be true	marks > 70 AND age < 23
\$or	At least one condition must be true	CSE OR ECE
\$not	Negates a condition	NOT (marks > 80)

Total dataset: 51 students (Roll No. 501-551).

Implementation Details

- The project is implemented using MongoDB and executed through the MongoDB Shell (mongosh).
- A single collection named "students" is used, with fields: rollNo, name, age, department and marks.
- insertOne() and insertMany() are used to populate the collection with 51 student records.
- updateOne() and updateMany() with the \$set operator are used to modify existing fields and add new ones (e.g. status).
- deleteOne() and deleteMany() are used to remove individual and bulk records based on a filter condition.
- find() is combined with comparison operators (\$gt, \$lt, \$eq, \$gte, \$ne) and logical operators (\$and, \$or, \$not) to retrieve specific subsets of student data.
- Each operation's acknowledgement object (acknowledged, matchedCount, modifiedCount, deletedCount) is used to verify that it executed as intended.

Results

All CRUD operations executed successfully on the students collection. The insertOne and insertMany operations populated the collection with 51 student records. The updateOne operation successfully changed one student's marks, while updateMany correctly modified all 13 CSE-department documents to add an "Active" status. The deleteOne operation removed one record and deleteMany found no records matching its filter, correctly reporting a deletedCount of 0. The query operations correctly filtered students by marks and department using comparison and logical operators, demonstrating that MongoDB's query engine returns accurate, condition-based results across a realistic-sized dataset.

Conclusion

This project highlights the practical use of MongoDB as a NoSQL database for managing structured academic data. By implementing insertion, update, deletion and a wide variety of query operations on a students collection, the project demonstrates a solid, hands-on understanding of MongoDB's Create, Read, Update and Delete (CRUD) capabilities as well as its comparison and logical query operators. The approach shows how a document database can be used to store and efficiently retrieve real-world tabular data such as student records.

Future Scope

- Use the MongoDB Aggregation Framework to compute department-wise averages, toppers and rankings.
- Add schema validation rules to enforce data types and required fields at the database level.
- Create indexes on frequently queried fields (e.g. marks, department) to improve performance.
- Build a front-end / web dashboard (e.g. using Node.js and Express) to interact with the database visually.
- Extend the collection with additional entities such as courses, attendance and results, and model relationships between them.

References

- MongoDB Official Documentation: <https://www.mongodb.com/docs/>
- MongoDB CRUD Operations: <https://www.mongodb.com/docs/manual/crud/>
- MongoDB Query Operators: <https://www.mongodb.com/docs/manual/reference/operator/query/>
- mongosh (MongoDB Shell) Documentation: <https://www.mongodb.com/docs/mongodb-shell/>

THANK YOU!